

TakeUForward — Master Project Plan & Technical Blueprint

Campus Peer-Mentorship & Community Platform

Working Document — Team of 2 · SSN College of Engineering

July 2026

Contents

1. Executive Summary	3
2. Problem Research & Discovery	3
2.1 How campus information actually flows today	3
2.2 Specific, named student pain points	3
2.3 Why existing solutions don't solve this	4
3. Vision Statement	4
4. Goals & Non-Goals	4
4.1 Goals	4
4.2 Non-Goals (v1 and near-term)	4
5. Personas	4
6. Full Feature Universe (All Phases)	5
6.1 Phase 1 — MVP (build first, fully functional, demo-ready)	5
6.2 Phase 2 — Near-term extensions (build after MVP is stable)	6
6.3 Phase 3 — Long-term / lower-priority utility features	7
7. Technology Stack — Final Decisions & Full Reasoning	8
7.1 Backend: Node.js + Express	8
7.2 Frontend: React (with Vite)	8
7.3 Database: MongoDB (NoSQL)	9
7.4 File Storage: AWS S3 (primary), Supabase Storage (fallback)	9
7.5 Authentication: Google OAuth, domain-restricted + alumni whitelist	9
7.6 Third-Party API: Google Gemini API	9
7.7 Notifications: Nodemailer (email) now, Web Push (Phase 2), WhatsApp (Phase 3, external-approval-gated)	9
7.8 Real-Time Chat: Polling (MVP) → Socket.io (Phase 2)	9
7.9 Deployment: Vercel (frontend) + Render (backend) + MongoDB Atlas + UptimeRobot	9
7.10 Testing: Playwright (E2E) + Jest (unit), run at the end of the build once the MVP is functionally complete	10
8. System Architecture	10
9. Authentication — Full Design	10
9.1 Student login (SSN-restricted Google OAuth)	10
9.2 Alumni login (personal email + pre-approval whitelist)	11
9.3 Club Admin accounts	11

10. Data Model (MongoDB / Mongoose Schemas)	11
11. Real-Time Chat — Full Explanation, MVP to Scale	13
11.1 Why this has a reputation for being “hard,” and why the MVP version isn’t	13
11.2 What WebSocket actually is, and how the Phase 2 upgrade works	13
11.3 Scaling beyond a single server (documented for the roadmap, not built at MVP stage)	13
12. Notifications	14
12.1 Email — Nodemailer (build now)	14
12.2 WhatsApp notifications — correctly identified as a good idea, but deferred, and here’s precisely why	14
12.3 Web Push Notifications (Phase 2)	14
13. Security & Anonymity Model — the platform’s most important design decision	14
13.1 Server-side identity stripping (technical enforcement)	14
13.2 Why anonymous 1:1 direct messages to strangers are explicitly NOT allowed in v1	14
13.3 Moderation	15
13.4 Rate limiting & general hardening	15
13.5 Error handling (explicit, since this is a working build document)	15
14. Club Content — Why Not Scrape Instagram, and the Better Alternative Chosen	15
15. Sub-Communities (Reddit-style structure)	15
16. API Surface (Core Routes)	15
17. Non-Functional Requirements	16
18. Build Plan — Minimal Working Version First, Then Layer Up	16
19. Success Metrics (for a real post-MVP launch)	17
20. Risks & Mitigations	17
21. Resume Bullets (honest, in-progress framing)	18
22. Appendix — Full Flat Feature List, All Phases	18

1. Executive Summary

TakeUForward is a full-stack campus community platform connecting juniors, seniors, and alumni for mentorship, academic resource sharing, placement guidance, and campus-wide announcements — replacing the fragmented mix of WhatsApp groups, Instagram DMs, and word-of-mouth that every engineering college currently relies on.

This document is the single working reference for building the project: the problem research behind it, every feature across every phase, the final technology stack with full reasoning for each choice, system architecture, complete data model, API design, the real-time chat implementation plan (from simple to advanced), authentication design, file storage explained in depth, notification systems, the anonymity/safety model, testing strategy, deployment plan, and a realistic build schedule.

Nothing in this document is abbreviated — this is the file to build from, end to end.

2. Problem Research & Discovery

2.1 How campus information actually flows today

At most Indian engineering colleges, including SSN, information relevant to a student’s academic and career trajectory is scattered across many disconnected channels simultaneously:

- **Department WhatsApp groups** — noisy, unsearchable, messages lost within days; seniors leave the group after graduating and take their knowledge with them
- **Instagram/LinkedIn DMs** — one-to-one, doesn’t scale; a junior has to already know *which specific senior* to even ask
- **Word of mouth / hostel corridor talk** — biased toward whoever is socially well-connected; introverted or new students are structurally disadvantaged
- **Club-specific Instagram pages** — event info scattered across many separate club handles, no unified calendar
- **CDC (Career Development Center) circulars / Google Forms / emails** — official but slow, one-directional, no peer commentary layer
- **Google Drive folders passed down informally** — notes/PYQs exist, but access depends on being in the “right” WhatsApp group at the right time
- **Random Telegram channels** — semi-anonymous, low-trust, frequently outdated

Core failure mode: there is no single, persistent, searchable, role-aware place where this information lives. Every incoming batch re-discovers the same mistakes and loses access to senior knowledge the moment that senior graduates.

2.2 Specific, named student pain points

Academic - Notes and PYQs exist somewhere, but only if you know the right senior - No structured way to know which elective/NPTEL course is actually worthwhile before choosing it - Assignment/internal-deadline visibility is scattered per class

Placement & Career - Company-specific interview experience (rounds, OA format, question patterns) is undocumented — it lives in a senior’s memory until they forget it or graduate - Referral requests happen through awkward, low-response-rate cold DMs to strangers - No visibility into “who from my college currently works at Company X” — this information exists but is invisible - Career-path confusion (SDE vs PM vs core vs higher studies) has no approachable, structured guidance layer

Social / Belonging - Fear of asking a “dumb question” publicly, especially across year/dept lines — no safe anonymous option exists in most current channels - Event/hackathon/workshop FOMO — students learn about great events only after they’ve happened - Seniors who *want* to mentor have no structured channel and end up fielding the same DM questions repeatedly

Operational (lower priority, but real) - Lost & found has no reliable channel - Secondhand book/cycle/calculator exchange happens ad hoc - Mess menu, bus timing changes, and similar utility updates are inconsistent

2.3 Why existing solutions don't solve this

- **Generic social media** (Instagram/LinkedIn): no tagging by year/dept/course, no anonymity option, content not meaningfully searchable
- **WhatsApp/Telegram groups**: ephemeral, unsearchable at scale, no role distinction, no persistence beyond active members
- **Official college portals**: one-directional, bureaucratic, no peer interaction layer
- **Fishbowl / Blind**: built for corporate employees, wrong audience, no academic/mentorship layer
- **LinkedIn**: too public/professional for casual doubt-asking, no anonymity, not campus-specific

Conclusion: a genuine, validated gap exists for a campus-native, role-aware, anonymity-optional, persistent knowledge and mentorship layer.

3. Vision Statement

“The single place a student needs to survive and thrive in college — connecting juniors with seniors and alumni for mentorship, centralizing academic and placement knowledge that would otherwise be lost year after year, and building an anonymous-safe space for honest questions — so no student has to rely on being in the ‘right’ WhatsApp group to get ahead.”

4. Goals & Non-Goals

4.1 Goals

1. Centralize academic, placement, and event/community information into one searchable, persistent, role-aware feed
2. Enable low-friction senior-to-junior knowledge transfer, with anonymity as an opt-in trust mechanism, not a forced default
3. Make alumni discoverable in a structured way for referrals and company-specific guidance, replacing cold DMs
4. Reduce FOMO — ensure no student misses an event, deadline, or opportunity because it was only posted in one scattered place
5. Give clubs a real, owned publishing channel inside the platform instead of relying on external social media

4.2 Non-Goals (v1 and near-term)

- Not a full LMS — does not replace the official college academic portal
 - Not a generic algorithmic social network — no engagement-maximizing feed manipulation
 - Not a payments platform — marketplace features (Phase 3) stay contact-exchange/barter only
 - Not multi-college in v1 — single-institution (SSN) first; data model designed to extend later
 - Not scraping external social media (see §14 for why, and the better alternative chosen instead)
-

5. Personas

Persona	Core need	Key frustration today
Junior (1st/2nd year)	Notes, roadmap guidance, event visibility, a safe space to ask “obvious” questions	Doesn’t know who to ask, afraid of judgment, missing context seniors take for granted
Senior (3rd/4th year)	Wants to mentor and give back, build reputation, without being spammed 1:1	No structured channel — ends up answering the same DM questions repeatedly
Alumni	Wants light-touch involvement — occasional referrals, AMAs — not constant obligation	No discoverability; current juniors don’t know how to reach them at all
Club Admin / CDC	Needs to broadcast events/updates fast, to the right audience, with ownership over their page	Managing large WhatsApp groups or a shared Instagram handle is unreliable and noisy

6. Full Feature Universe (All Phases)

6.1 Phase 1 — MVP (build first, fully functional, demo-ready)

#	Feature	Description
1	Auth (Google OAuth, domain-restricted + alumni whitelist)	Students sign in with SSN institutional Google account only; alumni sign in with personal Gmail, permitted only if pre-approved (see §9)
2	Role & Profile System	Every user has a clear, visible role: Student (tagged year + dept), Alumni (tagged current company, verified badge), Club Admin
3	Sub-Community Feed Structure	Reddit-style sub-communities: dept-wise (CSE-2028), batch-wise, and a general campus-wide feed; posts belong to exactly one community
4	Discussion / Q&A Posts	Post questions/discussions inside a community, tagged by course code where relevant; identified or anonymous, chosen per post
5	Anonymity Engine (posts + safe-mode replies)	Server-side identity stripping for anonymous posts/comments — never sent to the client at all. Anonymous 1:1 DMs to strangers are not permitted (see §13 for the safety reasoning); anonymous replies are allowed only in the context of an existing post thread

#	Feature	Description
6	Comments & Upvotes	Threaded comments on posts, independently anonymous or identified from the parent post
7	Academic Resource Repository	Upload/download notes, PYQs, and study material, tagged by course code + semester, stored in cloud object storage
8	Club Pages (self-managed, not scraped)	Each club gets an admin-managed page inside the platform to post events/announcements directly — the platform-native replacement for relying on Instagram (see §14)
9	Announcements Feed	Aggregated view of all club/CDC posts, filterable by category (event, placement, hackathon, workshop)
10	1:1 Direct Messaging (identified, polling-based MVP)	Simple database-backed messaging — see §11 for the full explanation of how this works and how it evolves
11	Report / Moderation Queue	Any post/comment reportable; auto-hide after a report-count threshold, pending manual review
12	Search & Filter	Filter feed/resources by community, dept, year, course code, tag, or post type
13	Email Notifications (Nodemailer)	Notify on: reply to your post, comment reply, mention, new post in a followed community
14	@Mentions	Tag a specific user in a post/comment; triggers a notification

6.2 Phase 2 — Near-term extensions (build after MVP is stable)

#	Feature	Description
15	Alumni Directory & Structured Referral Board	A junior posts “targeting Company X”; matched against alumni tagged with that company — replaces cold DMs entirely
16	Course & Professor Reviews	Rate/review electives and professors before registration
17	Mock Interview / Resume Review Pairing	Seniors who’ve cleared a specific company paired with juniors targeting that same company
18	Real Real-Time Chat (WebSocket/Socket.io upgrade)	Upgrades the Phase 1 polling-based chat to instant, push-based delivery — see §11 for the full technical explanation of this migration

#	Feature	Description
19	Trending / Hot Sort	Feed sort option beyond “newest,” weighted by recent upvotes/comment velocity
20	Verified Alumni Badge	Visual badge for confirmed, whitelisted alumni accounts
21	Web Push Notifications	Browser push as a lighter-weight alternative to email for logged-in, opted-in users
22	Personal Tracker / Bookmarks	Save posts, resources, and deadlines to a personal dashboard
23	NPTEL / Elective Suggestion Aggregator	Crowdsourced senior recommendations on which electives/NPTEL courses are worthwhile
24	Career Roadmap Templates	Structured guides per career path (SDE, PM, core, higher studies) crowdsourced from alumni
25	Weekly Digest Email	Auto-compiled summary of top posts, new resources, and upcoming events

6.3 Phase 3 — Long-term / lower-priority utility features

#	Feature	Description
26	WhatsApp Notifications	Deferred — requires Meta/Twilio WhatsApp Business API approval and template pre-approval, an external process outside the team’s control (see §12)
27	Lost & Found Board	Post/search lost items with location tagging
28	Secondhand Marketplace	Buy/sell/exchange books, cycles, calculators (contact-exchange only, no in-app payments)
29	Roommate / Hostel-Room Finder	Compatibility-based matching for incoming juniors
30	Shared Deadline/Assignment Tracker	Class-wide aggregated view of upcoming deadlines
31	Confession / Rant Board	Fully anonymous, high-engagement — requires the strongest moderation safeguards of any feature
32	Mess Menu / Bus Timing Updates	Small daily-utility updates — explicitly deprioritized, high-retention if added later
33	Multi-College Expansion	College/tenant field added to the data model, enabling expansion beyond SSN

7. Technology Stack — Final Decisions & Full Reasoning

This section documents not just the choice, but *why*, evaluated against every real alternative, assuming no prior backend/frontend experience (per team’s explicit request).

7.1 Backend: Node.js + Express

Alternative considered	Why not chosen
Next.js API routes	Adds file-based routing + server/client component conceptual overhead not needed for a login-gated dashboard; critically, Vercel’s serverless functions don’t support persistent WebSocket connections well, so a separate real-time server would be needed anyway, defeating the “one repo” convenience
Spring Boot (Java)	More verbose for a from-scratch learner; WebSocket setup (STOMP) carries more ceremony than Socket.io; not chosen since the team is explicitly starting fresh with no backend-language bias
Django / FastAPI (Python)	Legitimate alternative, fast to write — but MongoDB integration is less idiomatic/common than the Node+Mongo pairing, and there’s less community material for this exact combination

Why Node + Express wins: same language as the frontend (JavaScript/TypeScript) — no context-switching while learning both ends simultaneously. Pairs naturally with MongoDB (Mongoose ODM was essentially built for this — the “MERN” stack: MongoDB, Express, React, Node — is the most tutorialized, most AI-training-data-covered full-stack combination that exists, meaning AI-assisted development (Copilot/Gemini/Codex) will produce fewer subtle bugs here than in a less common combination.

7.2 Frontend: React (with Vite)

Alternative considered	Why not chosen
Vue	Gentler learning curve, excellent docs — but smaller ecosystem, and Indian recruiter/ATS keyword-scanning favors “React” far more often
Angular	Heavier, steeper, more boilerplate than needed for this scope
Svelte/SvelteKit	Less code to write — but smallest community of the options, meaning less AI training data and more time debugging AI-generated code that doesn’t quite work
Plain HTML/CSS/JS	Fastest to start, but manually reinvents state management (auth token handling, live feed re-rendering) that React provides — more total code, and skips the “React” resume keyword every relevant JD asks for

Why React wins: largest ecosystem, most job-description keyword recognition, and the most AI-training-data-covered frontend framework — fewer AI-hallucination bugs while learning it live, alongside Node on the backend.

7.3 Database: MongoDB (NoSQL)

Where SQL would actually be fine: the `users` collection — fixed fields (name, email, year, dept) — a relational table would work perfectly well here, no argument.

Why NoSQL wins overall: `posts`, `comments`, and `chat messages` have variable shape by type (a question post vs. an event post vs. a resource post carry different fields), and comments/messages naturally nest inside their parent — in MongoDB this is one document with an embedded array; in SQL it's a separate table with a foreign key and a join on every single read. This is a genuine structural fit, not an arbitrary preference — plus it was already a stated project requirement.

7.4 File Storage: AWS S3 (primary), Supabase Storage (fallback)

How S3 actually works, explained fully:

- Files live in **buckets** (top-level containers, like a root folder) as **objects**, each addressed by a **key** (essentially a file path string, e.g. `resources/cse-2028/os-notes.pdf`).
- The secure, standard upload pattern is: the backend (Node/Express) asks AWS for a **presigned URL** — a temporary, permission-scoped link valid for a short window (e.g., 5 minutes) that grants upload rights to one specific object key. The frontend then uploads the file **directly to S3** using that URL — the file's bytes never pass through your own backend server, keeping it lightweight and avoiding a slow upload bottleneck.
- For downloads, the same presigned-URL pattern (or a public-read bucket policy, if the resource is meant to be openly shared) lets the frontend fetch the file directly from S3's CDN-backed edge, not through your server.
- AWS's free tier includes roughly 5GB storage and a few thousand requests/month, but **only for an account's first 12 months**, and requires a card on file (no charge while within the free tier, but it exists as a safeguard).

Fallback plan — Supabase Storage: functionally the same idea (S3-compatible object storage under the hood), with a simpler SDK, no separate AWS IAM account/policy setup, and genuine prior team familiarity from earlier projects. **Decision:** attempt AWS S3 first since it is the more universally recognized “cloud” keyword for a resume line; if bucket policy/IAM setup consumes disproportionate time mid-build, switch to Supabase Storage without hesitation — both are honestly describable as “S3-compatible object storage” in an interview.

7.5 Authentication: Google OAuth, domain-restricted + alumni whitelist

Full flow explained in §9.

7.6 Third-Party API: Google Gemini API

Used narrowly and honestly: (a) minimal chatbot for platform-navigation/FAQ-style queries, and (b) summarization of uploaded academic resources. Kept intentionally scoped — not over-engineered — per explicit direction.

7.7 Notifications: Nodemailer (email) now, Web Push (Phase 2), WhatsApp (Phase 3, external-approval-gated)

Full reasoning in §12.

7.8 Real-Time Chat: Polling (MVP) → Socket.io (Phase 2)

Full technical explanation in §11.

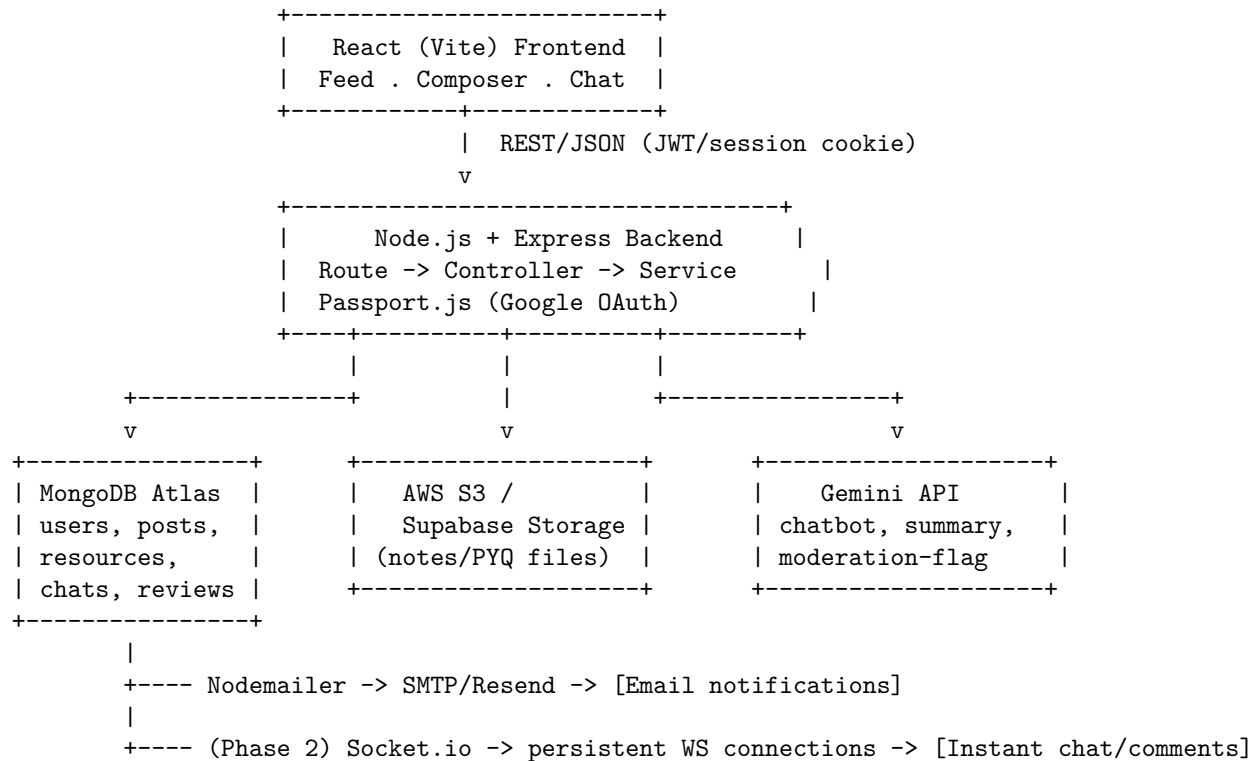
7.9 Deployment: Vercel (frontend) + Render (backend) + MongoDB Atlas + UptimeRobot

Render's free tier spins a service down after roughly 15 minutes without traffic, causing a 30–50 second cold-start delay on the next request. **UptimeRobot** (or `cron-job.org`) pinging the backend every 5–10 minutes

is a well-known, standard workaround. Worth noting for awareness: this works against Render's cost-saving intent, so it's normal for hobby/demo projects but not something to rely on for genuine production traffic.

7.10 Testing: Playwright (E2E) + Jest (unit), run at the end of the build once the MVP is functionally complete

8. System Architecture



Deployment topology: Frontend (Vercel) <-> Backend API (Render, kept warm via UptimeRobot) <-> MongoDB Atlas + AWS S3/Supabase + Gemini API (all external managed services — no self-hosted infrastructure to maintain).

9. Authentication — Full Design

9.1 Student login (SSN-restricted Google OAuth)

- Implemented via Passport.js's Google OAuth 2.0 strategy in Express.
- On the Google consent screen, request the `hd` (hosted domain) parameter set to the SSN email domain — this restricts the Google account picker to only institutional accounts at the identity-provider level.
- On the backend callback, **re-verify** the returned email's domain server-side regardless of the `hd` parameter (client-side/URL parameters can be tampered with — never trust them alone).
- On first login, create a `users` document tagged with role `student`, parsing year/dept from the email or a one-time profile-completion form.

9.2 Alumni login (personal email + pre-approval whitelist)

- Maintain a separate `approvedAlumniEmails` collection: `{ email, invitedBy, currentCompany, status: pending/verified }`.
- When a Google login arrives from a non-SSN domain, check this collection before granting access. If absent, deny with a clear “request alumni access” path instead of a silent failure.
- **Closing the “who verifies alumni” gap:** use an **invite-link system** — an already-verified alumnus or an admin (the founding team, initially) generates a one-time invite link containing a signed token; opening that link and completing Google OAuth pre-registers the email into `approvedAlumniEmails` with `status: verified` before their first real login attempt. This avoids anyone being able to falsely claim alumni status.

9.3 Club Admin accounts

- Manually provisioned by the founding team initially (small, known set of clubs) — a `role: club_admin` flag tied to a specific `clubId`, giving posting rights to that club’s page only.
-

10. Data Model (MongoDB / Mongoose Schemas)

```
// users
{
  _id, name, email, googleId,
  role: ["student" | "alumni" | "club_admin"],
  year: Number,           // students
  dept: String,           // students
  currentCompany: String, // alumni
  isVerifiedAlumni: Boolean,
  clubId: ObjectId,       // club_admin only
  bio: String,
  isAnonymousDefault: Boolean,
  reputation: Number,
  createdAt
}

// approvedAlumniEmails
{
  _id, email, invitedBy: ObjectId, currentCompany: String,
  status: ["pending" | "verified"], inviteToken, createdAt
}

// communities
{
  _id, name,           // e.g. "CSE-2028", "general", "placements"
  type: ["dept" | "batch" | "general" | "topic"],
  description, memberCount, createdAt
}

// posts
{
  _id, authorId,           // stripped from API response if isAnonymous = true
  communityId,
  isAnonymous: Boolean,
```

```

    type: ["question" | "resource" | "announcement" | "event"],
    tags: { dept, year, courseCode },
    content: String,
    mentions: [userId],           // @mention targets
    comments: [
      { authorId, isAnonymous, text, createdAt }
    ],
    upvotes: [userId],
    reports: [{ userId, reason, createdAt }],
    createdAt
  }

// resources
{
  _id, uploaderId, courseCode, semester,
  fileUrl,           // S3 / Supabase Storage object URL
  title, tags: [String],
  aiSummary: String, // Gemini-generated
  upvotes: [userId],
  createdAt
}

// clubs
{
  _id, name, description, adminIds: [userId],
  posts: [postId],      // or queried by community/tag instead
  createdAt
}

// chats
{
  _id, participants: [userId, userId],
  messages: [
    { senderId, text, readAt, createdAt }
  ]
}

// notifications
{
  _id, userId, type: ["reply" | "mention" | "comment" | "digest"],
  refId,           // related post/comment ID
  isRead: Boolean, createdAt
}

// reviews (Phase 2)
{
  _id, courseCode, professorName, authorId, isAnonymous,
  rating: Number, semester, comment, createdAt
}

// referralRequests (Phase 2)
{
  _id, requesterId, targetCompany,
  status: ["open" | "matched" | "closed"],

```

```
    matchedAlumniId, createdAt
  }
```

11. Real-Time Chat — Full Explanation, MVP to Scale

11.1 Why this has a reputation for being “hard,” and why the MVP version isn’t

Normal HTTP request/response (what you already understand from any API call) works like this: the client asks a question, the server answers, and the connection closes. To check for something *new*, you have to ask again. This repeated asking is called **polling**, and it requires **zero new concepts** — just a `setInterval` on the frontend calling your existing “get messages” REST endpoint every few seconds.

This is the MVP approach, and it is completely legitimate, not a hack: 1. User sends a message → `POST /api/chats/:id/message` → saved to the `chats` document in MongoDB, exactly like any other database write you already know how to do. 2. The frontend, on a timer (e.g., every 3–5 seconds) or on window focus, calls `GET /api/chats/:id` → renders whatever messages exist. 3. That’s the entire mechanism. No WebSockets, no persistent connections, no new infrastructure.

The trade-off: a few seconds of delay before a new message appears, and some wasted requests when nothing’s changed. For a demo/MVP with a small user base, this is a completely reasonable, honest engineering decision — and a good, real answer if an interviewer asks “why not WebSocket from day one”: *“I started with polling to validate the feature and ship fast; WebSocket is the natural next optimization once latency actually matters.”*

11.2 What WebSocket actually is, and how the Phase 2 upgrade works

A **WebSocket** is a single, persistent, two-way connection between client and server that stays open, instead of opening a fresh connection per request. Either side can push data through it at any time, instantly, without the other side having to ask first.

How it works underneath, in Node specifically, using Socket.io (the standard library — you don’t implement the raw protocol yourself): - The server keeps a live in-memory map of `userId` → `active socket connection`. - When user A sends a message: the server (1) saves it to MongoDB — **identical** to the polling approach, this part never changes — and (2) if user B currently has an open socket connection, immediately pushes the message to B via `socket.emit()`. If B is offline, the message simply waits in the database until their next login/reconnect, exactly like an SMS queued for a phone that’s off. - On the frontend, instead of a polling timer, you have a persistent `socket.on('newMessage', ...)` listener that updates the UI the instant a push arrives.

Migration path: because both approaches write to the *same* `chats` collection, upgrading from polling to Socket.io later doesn’t change your data model at all — you’re only replacing *how the frontend learns about new messages exist*, not how they’re stored. This is precisely why building polling first is a safe, non-wasted first step.

11.3 Scaling beyond a single server (documented for the roadmap, not built at MVP stage)

If the app ever runs on multiple backend server instances, a plain in-memory `userId` → `socket` map breaks, because user A and user B might be connected to *different* server instances that don’t know about each other. The standard fix is a **Redis adapter for Socket.io**, which lets all server instances share connection state and broadcast messages to the correct instance regardless of which one a user is connected to. This is a well-documented, standard pattern — worth knowing about and mentioning as the “how would this scale” answer, but not something to build for an MVP.

12. Notifications

12.1 Email — Nodemailer (build now)

Straightforward given the Node backend: Nodemailer sends email via SMTP — Gmail SMTP is fine for low volume during development, with a transactional provider (Resend, SendGrid) as a more reliable option if volume grows. Triggers: reply to your post, comment on your thread, @mention, weekly digest (Phase 2).

12.2 WhatsApp notifications — correctly identified as a good idea, but deferred, and here’s precisely why

Real WhatsApp Business API access (via Meta directly or through Twilio) requires **business account verification and message-template pre-approval by Meta** — an external review process with a timeline outside the team’s control, plus per-message costs beyond a small sandbox tier. This is not a skill gap the team can close by learning something; it is an external gatekeeping process. **Decision:** ship email notifications now, explicitly list WhatsApp integration as a Phase 3 roadmap item with this exact caveat documented — an honest, mature scoping decision rather than a silent drop.

12.3 Web Push Notifications (Phase 2)

A lighter-weight alternative to email for users who are logged in and have opted in — uses the browser’s Push API + a service worker; no external approval process required, unlike WhatsApp, making it a genuinely buildable Phase 2 item.

13. Security & Anonymity Model — the platform’s most important design decision

13.1 Server-side identity stripping (technical enforcement)

When a post or comment has `isAnonymous: true`, the `authorId` field **must be removed from the response object in the backend service layer before it is ever serialized and sent to the client** — never merely hidden via frontend conditional rendering or CSS. This means there is no code path, no browser dev-tools inspection, no API response, in which an anonymous author’s identity is exposed. This is the single most defensible engineering decision in the project and should be the first thing built and tested (write the test for it immediately, not as an afterthought).

13.2 Why anonymous 1:1 direct messages to strangers are explicitly NOT allowed in v1

This is a genuine safety distinction, not just a technical one. **Anonymous public posts** are relatively safe because they are visible to the whole community and moderatable — a pattern of abuse is noticeable and reportable by anyone who sees it. **Anonymous private 1:1 messages** are a fundamentally different risk category: they are unmoderated by anyone except the two participants, and could let one person harass a specific individual with zero accountability and no way for the community or admins to ever notice the pattern.

The safer alternative implemented instead: a user can reply *anonymously in the context of a post they authored* — so there is still a server-side record of who is behind the reply (for moderation purposes, never exposed to the recipient) — rather than opening a cold, fully anonymous DM channel to any arbitrary user. Regular 1:1 chat stays identified by default, with a report/block action always available.

13.3 Moderation

Posts/comments crossing a report-count threshold are auto-hidden pending admin review — preventing abuse of the anonymity feature without requiring constant manual moderation from day one.

13.4 Rate limiting & general hardening

Rate-limit post/comment/message creation per user to prevent spam floods (using a middleware like `express-rate-limit`). Validate and sanitize all user input server-side (not just client-side) to prevent injection-style attacks. Store OAuth tokens/session data securely (httpOnly cookies, not localStorage, to reduce XSS token-theft risk).

13.5 Error handling (explicit, since this is a working build document)

- Every Express route wrapped in consistent try/catch with a centralized error-handling middleware, returning structured JSON errors (`{ error: { code, message } }`) rather than leaking stack traces to the client.
- Mongoose validation errors, JWT/session failures, and file-upload failures each mapped to clear, distinct HTTP status codes (400, 401, 413, 500) rather than a generic catch-all.
- Frontend: a global API-error interceptor (Axios interceptor or equivalent) surfacing user-friendly messages rather than raw error objects.

14. Club Content — Why Not Scrape Instagram, and the Better Alternative Chosen

Scraping club Instagram pages has two real problems, not merely a difficulty problem: 1. It violates Instagram's Terms of Service and risks the scraping account/IP being blocked. 2. Instagram's official Graph API only grants an app access to accounts that have explicitly connected and authorized *that specific app* — it does not allow pulling arbitrary third-party public pages' content programmatically without their cooperation. It is also technically fragile, breaking whenever Instagram changes its page structure.

Chosen alternative: each club is given an **admin-managed page inside the platform itself** (`clubs` collection, §10), where designated club admins post events/announcements directly. This is ToS-safe, robust, gives clubs real ownership of their content, and — as a genuine product improvement, not just a workaround — makes the platform the authoritative source of truth instead of a fragile mirror of an external service the team doesn't control.

15. Sub-Communities (Reddit-style structure)

Formalizes what would otherwise be tag-based filtering into visually distinct spaces. A `communities` collection (e.g., `CSE-2028`, `general`, `placements`, plus per-club communities) that posts reference by `communityId`. Membership in dept/batch communities is auto-assigned at signup based on the user's `dept/year`; topic communities (e.g., `placements`, `hackathons`) are open-join. This is a clean, natural extension of the existing data model — not a redesign — since posts already carry a `communityId` field.

16. API Surface (Core Routes)

GET `/api/auth/google` -> redirects to Google OAuth consent

```

GET    /api/auth/google/callback    -> verifies domain/whitelist, issues session
POST   /api/auth/alumni/invite     -> admin/verified-alumni generates invite link
GET    /api/auth/logout

GET    /api/communities
GET    /api/communities/:id/posts

GET    /api/posts?community=&type=&dept=&year=&courseCode=
POST   /api/posts
GET    /api/posts/:id
POST   /api/posts/:id/comment
POST   /api/posts/:id/upvote
POST   /api/posts/:id/report

GET    /api/resources?courseCode=
POST   /api/resources           (presigned S3/Supabase upload, then metadata saved)

GET    /api/clubs/:id
POST   /api/clubs/:id/posts      (club_admin only)

GET    /api/chats/:userId
POST   /api/chats/:userId/message (polling MVP; emitted via Socket.io in Phase 2)

GET    /api/notifications
POST   /api/notifications/:id/read

-- Phase 2 --
GET/POST /api/reviews?courseCode=
GET/POST /api/referralRequests
POST     /api/referralRequests/:id/match

```

17. Non-Functional Requirements

- **Testing:** Jest unit tests covering the auth flow, the anonymity-stripping logic specifically (a dedicated test proving `authorId` never appears in an anonymous post's serialized response), and post/resource CRUD. One Playwright end-to-end test covering the critical path: Google login → post a question anonymously in a community → a second user comments → resource upload → search by course code. Run this testing pass **at the end**, once the MVP is functionally complete, per the team's stated build order. Report an actual coverage percentage in the README.
 - **Performance:** paginate all feed/resource list endpoints; index MongoDB collections on frequently filtered fields (`communityId`, `dept`, `year`, `courseCode`, `type`).
 - **Error handling:** see §13.5.
 - **Scalability (documented for the roadmap):** MongoDB's document model and a stateless Express API layer allow horizontal scaling behind a load balancer if usage grows; the Redis-adapter pattern (§11.3) documents the real-time chat scaling path.
-

18. Build Plan — Minimal Working Version First, Then Layer Up

Ownership split across a team of 2 to avoid merge conflicts; sync every 4–6 hours to integrate.

Person A: Auth (Google OAuth + whitelist) + Posts/Comments/Anonymity engine + Communities **Person B:** Resources (S3 upload) + Chat (polling MVP) + Club pages + Gemini integration + Nodemailer

Stage	Person A	Person B
Setup	Repo, MongoDB Atlas connected, skeleton deployed live immediately (Vercel + Render)	Same — pair on initial setup
Core Auth	Passport.js Google OAuth, domain restriction, alumni whitelist collection	Frontend skeleton (React/Vite) — login page, feed layout shell
Core Content	Posts CRUD + comments + upvotes + anonymity-stripping logic (write its test immediately) + Communities collection/routing	Resource upload flow (S3 presigned URL) + course-code tagging + filter UI
Secondary Features	Report/moderation queue + rate limiting + @mentions	Club pages (admin-post flow) + Announcements feed (reuse posts model, type: announcement)
Integrations	Notifications collection + trigger logic	Nodemailer email sending + Gemini API integration (chatbot/summarization)
Chat	—	1:1 chat, polling-based MVP (§11.1)
Testing & Polish	Jest tests (auth, anonymity, CRUD)	Playwright E2E on the critical path
Finalize	README (architecture diagram + roadmap section) + deploy stability pass	Joint bug-fixing pass, UptimeRobot configured for Render

19. Success Metrics (for a real post-MVP launch)

- Weekly active posters and commenters (engagement depth, not just signups)
- % of posts using anonymity (validates the trust-mechanism hypothesis)
- Resource repository size and download counts (validates the knowledge-hoarding-fix hypothesis)
- Referral board match rate (Phase 2)
- Report-to-hide ratio (moderation health check)

20. Risks & Mitigations

Risk	Mitigation
Aggressive build timeline for full feature set	MVP scope deliberately cut to 14 features; Phases 2/3 explicitly deferred and documented, not silently dropped
Anonymity feature could be abused for harassment	Report-threshold auto-hide + rate limiting + moderation queue built in from day one; anonymous 1:1 DMs to strangers explicitly disallowed (§13.2)

Risk	Mitigation
MongoDB’s schema flexibility could lead to inconsistent post structures	Enforce shape validation per <code>type</code> at the Mongoose schema/service layer, even though MongoDB itself is schemaless
Polling-based chat feels laggy at higher usage	Documented, low-risk upgrade path to Socket.io (§11.2) that requires no data model changes
AWS S3 IAM/bucket misconfiguration risk under time pressure	Supabase Storage fallback plan already identified (§7.4) if S3 setup consumes disproportionate time
Render free-tier cold starts	UptimeRobot keep-alive pings (§7.9)
Low initial adoption (cold-start problem common to all community platforms)	Seed the platform with the founding team’s own real notes/PYQs/mentorship content before wider launch
False alumni claims	Invite-link-based whitelist system (§9.2) rather than open self-registration for non-SSN emails

21. Resume Bullets (honest, in-progress framing)

Since this is an ongoing project, phrase in active/building tense rather than claiming full completion prematurely:

1. *“Architecting TakeUForward, a full-stack campus mentorship platform (React, Node.js/Express, MongoDB) enabling anonymous or identified peer-to-senior knowledge sharing across dept-wise and batch-wise sub-communities”*
2. *“Designed a server-side anonymity model that strips author identity at the API response layer, with a report-threshold moderation queue and a safety-first policy disallowing anonymous DMs to strangers”*
3. *“Implemented Google OAuth with domain-restricted student login and an invite-based alumni verification system; integrated AWS S3 for academic resource storage and the Gemini API for resource summarization”*

22. Appendix — Full Flat Feature List, All Phases

Feature	Phase
Google OAuth (SSN-restricted + alumni whitelist)	1
Role & Profile System	1
Sub-Community Feed Structure	1
Discussion/Q&A Posts	1
Anonymity Engine (posts + safe-mode replies)	1
Comments & Upvotes	1
Academic Resource Repository	1
Club Pages (self-managed)	1
Announcements Feed	1
1:1 Direct Messaging (polling)	1
Report/Moderation Queue	1
Search & Filter	1
Email Notifications (Nodemailer)	1
@Mentions	1
Alumni Directory & Referral Board	2

Feature	Phase
Course & Professor Reviews	2
Mock Interview / Resume Review Pairing	2
Real-Time Chat (Socket.io upgrade)	2
Trending / Hot Sort	2
Verified Alumni Badge	2
Web Push Notifications	2
Personal Tracker / Bookmarks	2
NPTEL / Elective Suggestion Aggregator	2
Career Roadmap Templates	2
Weekly Digest Email	2
WhatsApp Notifications	3
Lost & Found Board	3
Secondhand Marketplace	3
Roommate / Hostel-Room Finder	3
Shared Deadline/Assignment Tracker	3
Confession / Rant Board	3
Mess Menu / Bus Timing Updates	3
Multi-College Expansion	3

End of Document — TakeUForward Master Plan v2.0